

THE DOM, ON PAPER

Everything the lecture named and did not have time to list. This deck is not taught — it is looked up, in the lab, and in week 13 when there is no internet.

WHAT IS HERE

The catalogues: selecting, building, events, forms, DevTools.

WHAT IS NOT

Explanations. Those are in the lecture deck, with the demos.

HOW TO USE IT

Print it, or keep the PDF open beside the editor. The last slide is the one to read first when something is broken.

SELECTING

CALL	RETURNS	NOTES
<code>querySelector(sel)</code>	Element or <code>null</code>	First match in document order. Any CSS selector.
<code>querySelectorAll(sel)</code>	NodeList (static)	A snapshot. <code>forEach</code> works; <code>map</code> does not — spread it first.
<code>el.querySelector(sel)</code>	Element or <code>null</code>	Scoped to <code>el</code> . <code>:scope > li</code> for direct children.
<code>getElementById(id)</code>	Element or <code>null</code>	The <code>id</code> , not a selector. No <code>#</code> .
<code>getElementsByName...</code>	HTMLCollection (live)	<code>ClassName</code> and <code>TagName</code> . Re-answer every read.

TRAVERSING: DOWN AND ALONG

PROPERTY	GIVES YOU
<code>parentElement</code>	The element above. <code>parentNode</code> can also be the document.
<code>children</code> · <code>childElementCount</code>	Element children only. Live collection.
<code>childNodes</code>	Everything, including whitespace text nodes and comments.
<code>firstElementChild</code> · <code>lastElementChild</code>	What you usually mean by first and last.
<code>firstChild</code> · <code>lastChild</code>	Often the indentation in your file.
<code>nextElementSibling</code> · <code>previousElementSibling</code>	Along one level

TRAVERSING: UP, AND ASKING QUESTIONS

CALL	GIVES YOU
<code>closest(sel)</code>	Up until something matches, itself included. <code>null</code> if nothing does.
<code>matches(sel)</code>	<code>true</code> / <code>false</code> . Does this element match that selector?
<code>contains(other)</code>	Is <code>other</code> inside this one? Useful for outside-click handlers.
<code>isConnected</code>	Is this element in the document at all? The answer to “I built it and cannot see it”.

CREATING AND INSERTING

CALL

`document.createElement(tag)`

`document.createTextNode(s)`

`parent.append(a, b, 'text')`

`parent.prepend(x)`

`el.before(x) · el.after(x)`

`el.replaceWith(x)`

WHAT IT DOES

A new element, **not** in the document. Nothing shows until it is attached.

A text node. [append](#) accepts a plain string instead.

At the end. Several at once, strings allowed.

At the start.

As a sibling, in front of or behind.

Swap this element for another.

EMPTYING, REMOVING, COPYING

CALL

`parent.replaceChildren(...)`

`el.remove()`

`el.cloneNode(deep)`

The 1998 trio you will read in old code: `appendChild`, `removeChild` and `insertBefore(x, ref)`. One node each, parent required.

WHAT IT DOES

Empty it, and optionally refill it. **This is how `render()` starts.**

Take this element out. No parent needed.

A copy. `deep = true` copies the children. **Listeners are never copied.**

CONTENT: FOUR WAYS IN, AND ONLY TWO YOU SHOULD USE

PROPERTY	READS	WRITES	USE IT?
<code>textContent</code>	All text, hidden included	Text. Never parsed.	Yes — the default.
<code>innerText</code>	Text as rendered : CSS applied	Text, and forces a layout	Only for what the user sees.
<code>innerHTML</code>	The markup inside	Parsed as HTML	Never with text a user typed.
<code>insertAdjacentHTML(pos, s)</code>	—	Parses only the new fragment	Existing children survive.
<code>insertAdjacentText(pos, s)</code>	—	Text, at a position	Safe version of the row above.

`pos` is one of `beforebegin` · `afterbegin` · `beforeend` · `afterend`.

CLASSES, STYLES, ATTRIBUTES, DATA

CALL

`classList.add / remove / toggle / contains`

`classList.replace(a, b)`

`el.style.prop = v`

`getComputedStyle(el).prop`

`getAttribute / setAttribute / removeAttribute`

`hasAttribute(name)`

`el.dataset.thing`

`el.hidden = true`

NOTES

`toggle(name, force)` takes a boolean: one call for both directions.

Swap one for another.

Writes an **inline** style, which beats every stylesheet. For computed values only.

What is actually applied. `el.style` only reads inline.

The HTML attribute — what the file said.

Presence, for boolean attributes like `hidden`, `disabled`.

`data-thing`, both ways. `data-unit-price` is `dataset.unitPrice`.

A property with an attribute behind it. The empty state, in one line.

THE EVENTS YOU WILL ACTUALLY USE

EVENT	FIRES WHEN	WATCH OUT
<code>click</code>	Pointer or keyboard activation of a button or link	Enter and Space fire it on a real <code><button></code> . On a <code><div></code> they do not.
<code>submit</code>	A form is submitted, by button or by Enter	On the form , never on the button.
<code>input</code>	Every keystroke, paste and undo	What you want for live filtering.
<code>change</code>	Value committed: blur for text, immediately for checkbox and select	Two different timings under one name.
<code>keydown</code>	A key goes down. <code>event.key</code> is <code>'Enter'</code> , <code>'Escape'</code> , <code>'a'</code>	<code>keypress</code> is dead. Do not use it.
<code>focus</code> · <code>blur</code>	An element gains or loses focus	They do not bubble. Use <code>focusin</code> / <code>focusout</code> for delegation.
<code>pointerdown</code> / <code>pointerup</code> cover mouse, touch and pen in one API — prefer them over <code>mousedown</code> for anything draggable.		

ADDEVENTLISTENER, IN FULL

```
1 target.addEventListener(type, handler, options);
2
3 target.addEventListener('click', onClick, {
4   capture: false,    // true = hear it on the way DOWN
5   once: true,        // remove itself after the first time
6   passive: true,     // promises not to call preventDefault (scroll performance)
7   signal: controller.signal, // remove it by aborting the controller
8 });
9
10 // removing needs the SAME function reference:
11 target.removeEventListener('click', onClick);           // works
12 target.removeEventListener('click', () => {});           // does nothing at all
13
14 // one listener, removed later, without keeping the reference:
15 const c = new AbortController(); /* ... */ c.abort();
```

THE EVENT OBJECT

FIELD	WHAT IT IS
<code>type</code>	'click', 'submit' — useful when one handler serves several.
<code>target</code>	What was hit. Often deeper than you expect.
<code>currentTarget</code>	Who is listening. The element you attached to. Never changes.
<code>eventPhase</code>	1 capturing · 2 at the target · 3 bubbling.
<code>key</code> · <code>code</code>	Keyboard. <code>key</code> is the character, <code>code</code> is the physical key.
<code>preventDefault()</code>	Cancels what the browser would have done.
<code>stopPropagation()</code>	Stops other listeners above from hearing it. Rarely correct.
<code>defaultPrevented</code>	Did someone already call <code>preventDefault</code> ?
<code>composedPath()</code>	The full array of nodes the event travels through. Excellent for debugging.



DELEGATION, AS A RECIPE

```
1 container.addEventListener('click', (event) => {  
2   const hit = event.target.closest('[data-action="remove"]');  
3   if (!hit) return;           // the click was not for us  
4  
5   const id = hit.closest('li').dataset.id; // identity from the DOM  
6  
7   removeItem(id);             // change the data  
8   render();                   // redraw from the data  
9 });
```

ATTACH TO

A container **render()** empties but **never replaces** — the `<ul>` itself, or a wrapper.

DO NOT ATTACH TO

`document` by default. It works, and every click in the page now runs your code.

SEVERAL ACTIONS

One listener, `data-action` on each control, and a `switch`. Not one listener per action.

FORMS

CALL

`input.value`

`checkbox.checked` · `radio.checked`

`select.value` · `select.selectedIndex`

`form.elements.name` · `form.name`

`new FormData(form)`

`form.reset()`

`form.requestSubmit()`

`input.setAttribute('aria-invalid', 'true')`

`input.focus()`

NOTES

Always a string, including `type="number"`. Convert at the boundary.

Boolean property. The `checked` attribute is only the initial state.

The chosen option's value.

Reach a field by its `name` attribute.

Every named field at once. `Object.fromEntries(fd)` gives a plain object.

Back to the initial values. Does not fire `input`.

Submit as if a button was pressed — validation and the `submit` event included.

What a screen reader announces. Set it **and clear it**.

Where the user types next. The third part of failing well.

DEVTOOLS, BY PANEL

PANEL	ANSWERS	WORTH KNOWING
Elements	What is on the page right now?	Live-edit anything. <code>\$0</code> in the console is the selected element.
Styles / Computed	Which rule won?	Struck-through rules lost. Computed is the final number.
Console	What is this value?	<code>table · dir · group · count · time · trace · \$\$()</code>
Sources	What happened, step by step?	Breakpoints, conditional breakpoints, Watch, Call Stack, Event Listener Breakpoints.
Network	Did the file even arrive?	Status, type filter, and Disable cache — leave it ticked.
Application	What is stored?	localStorage lives here. Week 10.

► F12 or Cmd+Option+I · Ctrl/Cmd+Shift+C picks an element · Esc opens the console drawer inside any panel.

THE SIX STEPS, AND WHAT EACH ONE COSTS

STEP	THE QUESTION IT ANSWERS	WHAT GOES WRONG
1 · Reproduce	What exact sequence makes it happen?	“Sometimes it breaks” is not a bug report, even to yourself.
2 · Isolate	Which file, function and line?	Editing before locating. Two changes, no information.
3 · Hypothesise	“X is Y when it should be Z.”	A guess you cannot disprove. It cannot be tested, so it cannot be wrong.
4 · Verify	Is that sentence true?	Skipping straight to the fix. The bug survives and now the code is different.
5 · Fix	What is the cause?	Fixing the symptom — changing the id in the HTML instead of the typo in the selector.
6 · Confirm	Is it gone, and is anything beside it broken?	The step that disappears when time runs out.

WHERE TO LOOK, IN ORDER

#	IF...	LOOK AT
1	There is a red line in the console	It, first. Click the file:line link — it opens Sources on the line.
2	Nothing happens, and nothing errors	Is the handler running at all? One breakpoint on its first line.
3	The handler runs and does the wrong thing	What are the values? Watch, not <code>console.log</code> .
4	What is on screen disagrees with the data	<code>console.table(items)</code> , and count the rows.
5	It works sometimes	What is different about the times it does? Usually <code>event.target</code> .
6	It worked and then stopped	Did something re-render? Every element is new after that.
7	Your change had no effect at all	Network: did the file load? Is Disable cache ticked?

► **THIS IS THE SLIDE TO READ FIRST WHEN SOMETHING IS BROKEN**, and the five faults in the Debug Challenge are rows 1, 2, 3, 4 and 5.

